PREDICTIVE TEXT GENERATOR

1. Introduction

This project implements a **Predictive Text Generator**, which suggests the next possible word based on the text typed by the user. The system uses Natural Language Processing (NLP) and a **statistical n-gram language model** to analyze word sequences and generate predictions. The goal is to simulate simple autocomplete behavior — similar to messaging applications or smart keyboards.

The project includes:

1. A preprocessing module

The preprocessing module is responsible for cleaning and preparing the text data before it is used for training. It converts text to lowercase, removes punctuation, and splits the text into tokens (individual words). This step ensures that the dataset is consistent, noise-free, and ready for building the N-gram model. Preprocessing is essential because raw text contains many variations that can confuse the model.

2. An n-gram model builder

The N-gram model builder analyzes the cleaned text and constructs unigram, bigram, and trigram frequency dictionaries. These represent how often words and word sequences appear in the dataset. The model builder saves these counts into a .pkl file so they can be loaded quickly during prediction. This step forms the “brain” of the predictive text generator.

3. A prediction module

The prediction module takes the user’s typed text and generates the next possible word using the n-gram models. It follows a **backoff strategy**: it first checks trigrams, then bigrams, and finally unigrams if no sequence matches. This ensures the system always provides a prediction, even with minimal context. The module produces fast, real-time suggestions as the user types.

4. A Flask-based web application

The Flask backend handles the server-side operations, including loading the n-gram model and responding to prediction requests. When the front end sends text input via API, Flask processes it and returns a list of suggested next words. It also renders the main webpage for users. Flask connects all components and makes the model accessible through a web interface.

5. A clean, modern front-end UI

The project includes a responsive HTML/CSS interface where users can type text and instantly receive suggestions. Suggestions appear as clickable "chips," making the experience interactive and user-friendly. The design is modern and visually appealing, offering smooth real-time interaction. This UI helps demonstrate the predictive model in a practical, easy-to-use format.

2.OBJECTIVES

The main objectives of the project are:

- To build a text generator using an **N-gram model**
- To predict the next word using **context awareness**
- To create a **real-time predictive UI** using Flask
- To train the model on sample corpus data
- To demonstrate NLP workflow, model building, and deployment

3. DATASET DESCRIPTION

A small custom dataset (sample_corpus.txt) was used to train the language model. Example lines include:

- *"I love programming in python"*
- *"Python is great for data science"*

4. SYSTEM ARCHITECTURE

4.1 Preprocessing

- Converts text to lowercase
- Removes punctuation
- Tokenizes text into words

4.2 N-gram Model Building (Training Phase)

Implemented in build_model.py:

- Unigrams (single-word frequencies)
- Bigrams (two-word sequences)
- Trigrams (three-word sequences)

These are stored in a .pkl model file for fast loading.

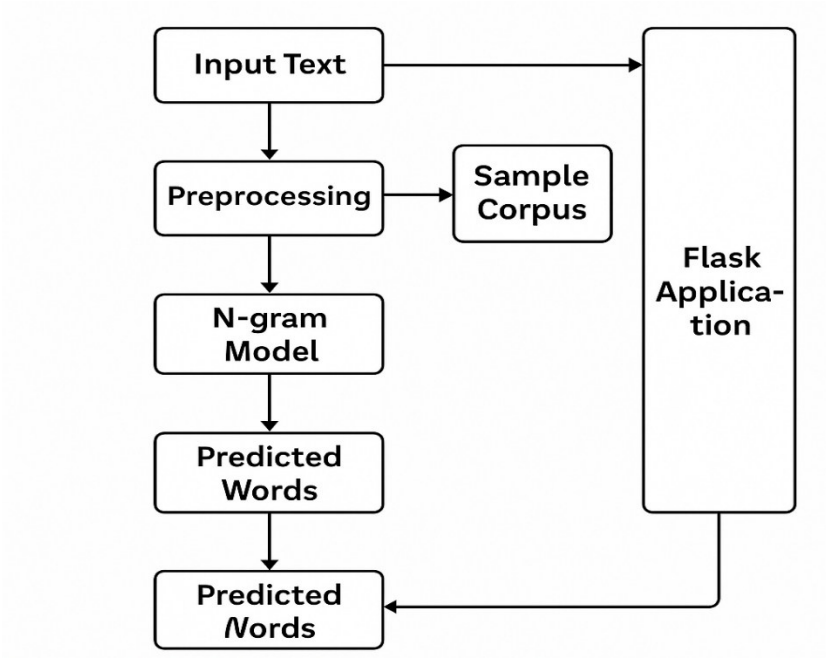
4.3 Prediction (Backoff Strategy)

Implemented in predict.py:

1. Try **trigram** prediction
2. If unavailable, try **bigram** prediction
3. If still unavailable, return most common **unigrams**

This backoff method ensures prediction is always generated.

Architecture Diagram



The architecture of the Predictive Text Generator consists of several integrated components that work together to deliver accurate next-word predictions. The process begins with **Input Text**, which the user types into the interface. This input is passed to the **Preprocessing module**, where the text is cleaned, converted to lowercase, and tokenized to prepare it for analysis. At the same time, a **Sample Corpus** is preprocessed and used to train the **N-gram Model**, which generates unigram, bigram, and trigram frequency patterns. During prediction, the user's

processed input is fed into the **Prediction Module**, where the n-gram model applies a backoff strategy—checking trigram, then bigram, and finally unigram patterns—to determine the most likely next words. These **Predicted Words** are then returned to the front end through the **Flask Application**, which acts as the communication bridge between the backend model and the user interface. Finally, the predictions are displayed as suggestions on the clean and modern UI, allowing users to click and insert them directly into their text. This architecture ensures smooth, real-time, and context-aware word prediction.

Applications of the Predictive Text Generator

1. Autocomplete in Messaging Apps

This model can be used to suggest the next word while typing in chat applications like WhatsApp, Messenger, or SMS apps, making typing faster and easier.

2. Search Engine Query Suggestions

Search engines like Google use similar N-gram models to suggest queries as the user types, improving user experience and accuracy.

3. Text Editors and Documentation Tools

Tools like MS Word, Google Docs, and email clients use predictive text to help users write faster with fewer mistakes.

4. Assistive Technology for Accessibility

Predictive text models help people with disabilities type faster by reducing the number of keystrokes required.

5. Voice-to-Text Systems

Speech recognition systems use N-gram models to guess the next word based on previous words, improving transcription accuracy.

6. Chatbots and Virtual Assistants

Predictive text helps chatbots generate more human-like responses by predicting the next logical word in a conversation.

7. Educational Typing Tools

Typing tutors and language learning apps use predictive text to help learners improve grammar, vocabulary, and writing flow.

5. FRONT-END APPLICATION

A clean, interactive UI was built using HTML + CSS.

- Live suggestions update as the user types
- Suggestion chips appear under the text box
- Clicking a suggestion inserts it into the text area

6. BACK-END APPLICATION (FLASK SERVER)

app.py handles:

- Rendering the home page
- Accepting API requests (/api/predict)
- Returning JSON word predictions

7. TECHNOLOGIES USED

1. Python

Python is the primary programming language used to build this entire project. All modules—including preprocessing, model building, prediction logic, and the web server—are written in Python. Its simplicity and large collection of libraries make it ideal for NLP and machine learning tasks. Python acts as the foundation that connects all the components of the predictive text generator.

2. Flask (Backend Server)

Flask is used to create the backend web server that handles user requests and sends predictions. When the front end sends text input, Flask processes it, calls the prediction module, and returns the suggested words as JSON. It also renders the main webpage (index.html) for users to interact with. Flask essentially brings the predictive model to life by making it accessible through a browser.

3. NLTK (Natural Language Toolkit) for Preprocessing

NLTK is used for basic NLP preprocessing tasks such as tokenizing text, removing symbols, and preparing clean input for the model. It helps convert sentences into a structured form so that n-gram sequences can be built accurately. Without proper preprocessing, the model might misinterpret words due to inconsistent formatting. NLTK makes this step reliable and efficient.

4. NumPy

NumPy is used for efficient data handling and numerical operations within the project. Although the model is simple, NumPy improves the performance of tasks such as counting frequencies, sorting predictions, and managing arrays. It helps make the prediction process fast and smooth, especially when working with larger datasets. NumPy ensures the project runs efficiently during both training and prediction.

8. EVALUATION

According to the report file:

- Manual qualitative testing was performed
- Model quality improves with larger datasets
- Accuracy can be checked by splitting test/train data

9. LIMITATIONS

- Small dataset → limited vocabulary
- No smoothing technique implemented
- Cannot handle unseen words well
- Not a deep-learning model, so accuracy is limited

10. FUTURE IMPROVEMENTS

Suggested improvements include:

- Add Laplace smoothing
- Add spell correction
- Add user-personalized dictionary
- Replace model with LSTM / transformer model for large datasets

11. CONCLUSION

This project successfully demonstrates the fundamentals of NLP text prediction using N-gram modeling. It covers end-to-end development including preprocessing, training, prediction logic, UI design, and deployment through Flask. It serves as a strong foundational project for moving toward more advanced NLP models.

12.REFERENCES

[1] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. Pearson, 2023.

[2] C. D. Manning and H. Schütze, *Foundations of Statistical Natural Language Processing*. MIT Press, 1999.

[3] NLTK Project, “Natural Language Toolkit Documentation,” 2024. [Online]. Available: https://www.nltk.org/

[4] Python Software Foundation, “Python Documentation,” 2024. [Online]. Available: https://docs.python.org/

[5] Pallets Projects, “Flask Web Framework Documentation,” 2024. [Online]. Available: https://flask.palletsprojects.com/

[6] NumPy Developers, “NumPy Documentation,” 2024. [Online]. Available: https://numpy.org/doc/

[7] Google Research, “N-gram Language Modeling Papers,” 2024. [Online]. Available: https://research.google/pubs/

[8] Kaggle, “NLP Tutorials and Datasets,” 2024. [Online]. Available: https://www.kaggle.com/